Параметры и аргументы функций

1. Параметры и аргументы функций
2. Как работают параметры и аргументы функций?
3. Как работают параметры и возвращаемые значения функций?
4. Еще примеры
5. Тест
6. Ответы

Параметры и аргументы функций

Во многих случаях нам нужно будет передавать данные в вызываемую функцию, чтобы она могла с ними как-то взаимодействовать. Например, если мы хотим написать функцию умножения двух чисел, то нам нужно каким-то образом сообщать функции, какие это будут числа. В противном случае, как она узнает, что на что перемножать? Здесь нам на помощь приходят параметры и аргументы.

Параметр функции — это переменная, которая используется в функции, и значение которой предоставляет caller (вызывающий объект). Параметры указываются при объявлении функции в круглых скобках. Если их много, то они перечисляются через запятую, например:

```
void doPrint()
{
    std::cout << "In doPrint()" << std::endl;
}

// Это функция имеет один параметр типа int: a
void printValue(int a)
{
    std::cout << a << std::endl;
}

// Это функция имеет два параметра типа int: a и b
int add(int a, int b)
{
    return a + b;
}
```

Параметры каждой функции действительны только внутри этой функции. Поэтому, если printValue() и add() имеют параметр с именем a, то это не означает, что произойдет конфликт имен. Эти параметры считаются независимыми и никак не взаимодействуют друг с другом.

Аргумент функции — это значение, которое передается из caller-а в функцию и которое указывается в скобках при вызове функции в caller-е:

```
printValue(7); // Это аргумент функции printValue()
add(4, 5);
```

Обратите внимание, аргументы тоже перечисляются через запятую. Количество аргументов должно совпадать с количеством параметров, иначе компилятор выдаст сообщение об ошибке.

Как работают параметры и аргументы функций?

При вызове функции, все её параметры создаются как локальные переменные, а значение каждого из аргументов копируется в соответствующий параметр (локальную переменную). Этот процесс называется **передачей по значению**. Например:

```
// Это функция имеет два параметра типа int: a и b
// Функция возвращает int в аргумент caller
void printValues(int a, int b)
{
    std::cout << a << std::endl;
    std::cout << b << std::endl;
}

int main()
{
    printValues(8, 9); // здесь два аргумента: 8 и 9

    return 0;
}
```

При вызове функции printValues() аргументы `8` и `9` копируются в параметры `a` и `b`. Параметру `a` присваивается значение `8`, а параметру `b` — значение `9`.

Результат:

```
8
9
```

Как работают параметры и возвращаемые значения функций?

Используя параметры и возвращаемые значения, мы можем создавать функции, которые будут принимать и обрабатывать данные, а затем возвращать результат обратно в caller.

Например, простая функция, которая принимает два целых числа и возвращает их сумму:

```
#include <iostream>

// Функция add() принимает два целых числа в качестве параметров и возвращает их сумму
// Возвращает int в аргумент caller
int add(int a, int b)
{
    return a + b;
}

// Функция main() не имеет параметров
int main()
{
    std::cout << add(7, 9) << std::endl; // аргументы 7 и 9 передаются в функцию add()

    return 0;
}
```

При вызове функции add(), параметру `a` присваивается значение `7`, а параметру `b` — значение `8`. Затем функция add() вычисляет их сумму и возвращает результат обратно в main(). И тогда уже результат выводится на экран.

Результат выполнения программы:

```
15
```

Еще примеры

Рассмотрим еще несколько вызовов функций:

```
#include <iostream>

int add(int a, int b)
{
    return a + b;
}

int multiply(int c, int d)
{
    return c * d;
}

int main()
{
    std::cout << add(7, 9) << std::endl; // вызов функции add(): a = 7, b = 9, значит 8 + 9 = 15
    std::cout << multiply(24, 5) << std::endl; // вызов функции multiply(): c = 4, d = 5, значит 5 * 4 = 20

    // Мы можем передавать также выражения в качестве аргументов
    std::cout << add(2 + 3, 4 * 5) << std::endl; // вызов функции add(): a = 5, b = 20, значит 5 + 20 = 25

    // Или даже переменные с другими значениями
    int x = 4;
    std::cout << add(x, x) << std::endl; // будет 4 + 4

    std::cout << add(1, multiply(2, 3)) << std::endl; // будет 1 + (2 * 3)
    std::cout << add(add(1, 2), 3) << std::endl; // будет 1 + (2 + 3)

    return 0;
}
```

Результат выполнения программы:

```
15
20
25
8
7
6
```

С первыми двумя вызовами всё понятно.

В третьем вызове аргументами являются выражения, которые сначала нужно обработать. `2 + 3` => `5` и результат `5` присваивается переменной `a`. `4 * 5` => `20` и результат `20` присваивается переменной `b`. Результатом выполнения функции add(5, 20) является значение `25`.

Следующая пара относительно лёгкая для понимания:

```
int x = 4;
std::cout << add(x, x) << std::endl; // будет 4 + 4
```

Здесь уже `a` = `x` и `b` = `x`. Поскольку `x` = `4`, то add(`x`, `x`) = add(`4`, `4`). Результат — `8`.

Теперь рассмотрим вызов посложнее:

```
std::cout << add(1, multiply(2, 3)) << std::endl; // будет 1 + (2 * 3)
```

При выполнении этого **стейтмента** процессор должен определить значения параметров `a` и `b` функции add(). С параметром `a` всё понятно — мы передаем значение `1` (`a` = `1`). А вот чтобы определить значение параметра `b`, нам необходимо выполнить операцию умножения: multiply(`2`, `3`), результат — `6`. Затем add(`1`, `6`) возвращает число `7`, которое и выводится на экран.

Короче говоря:

add(`1`, multiply(`2`, `3`)) => add(`1`, `6`) => `7`

Последний вызов может показаться немного сложным из-за того, что параметром функции add() является другой вызов add():

```
std::cout << add(1, add(2, 3)) << std::endl; // будет 1 + (2 + 3)
```

Но здесь всё аналогично вышеприведенному примеру. Перед тем, как процессор вычислит внешний вызов функции add(), он должен обработать внутренний вызов функции add(`2`, `3`). add(`2`, `3`) = `5`. Затем процессор обрабатывает функцию add(`1`, `5`), результатом которой является значение `6`. Затем `6` передается в std::cout.

Короче говоря:

add(`1`, add(`2`, `3`)) => add(`1`, `5`) => `6`

Тест

Задание №1: Что не так со следующим фрагментом кода?

```
void multiply(int a, int b)
{
    return a * b;
}

int main()
{
    std::cout << multiply(7, 8) << std::endl;

    return 0;
}
```

Задание №2: Какие здесь есть две проблемы?

```
int multiply(int a, int b)
{
    int product = a * b;
}

int main()
{
    std::cout << multiply(5) << std::endl;

    return 0;
}
```

Задание №3: Какой результат выполнения следующей программы?

```
#include <iostream>

int add(int a, int b, int c)
{
    return a + b + c;
}

int multiply(int a, int b)
{
    return a * b;
}

int main()
{
    int doubleNumber(int a);
    int add(int, int);
    int multiply(int, int);

    int a = add(3, 4, 5);
    a = multiply(a, 2);
    a = doubleNumber(a);
    std::cout << a << std::endl;

    return 0;
}
```

Задание №4: Напишите функцию doubleNumber(), которая принимает целое число в качестве параметра, удваивает его, а затем возвращает результат обратно в caller.

Задание №5: Напишите полнофункциональную программу, которая принимает целое число от пользователя (используйте std::cin), удваивает его с помощью функции doubleNumber() из предыдущего задания, а затем выводит результат на экран.

Ответы

Ответ №1

Функция multiply() имеет тип возврата void, что означает, что эта функция не возвращает значения. Но, так как она все равно пытается возвратить значение с помощью оператора return, мы получим ошибку от компилятора. Функция должна иметь тип возврата int.

Ответ №2

Проблема №1: main() передает один аргумент в multiply(), но multiply() имеет два параметра.

Проблема №2: multiply() вычисляет результат и присваивает его локальной переменной, которую не во возвращает обратно в main(). А поскольку тип возврата функции multiply() — int, то мы получим ошибку (в некоторых компиляторах) или неожиданные результаты (в остальных компиляторах).

Ответ №3

Функция multiply() принимает следующие параметры: a = add(3, 4, 5) и b = 5. Сначала процессор обраба тывает a = add(3, 4, 5), т.е. 3 + 4 + 5 = 12. Затем уже выполняет операцию умножения, результатом которой является 60. Ответ: 60.

Ответ №4

```
int doubleNumber(int a)
{
    return 2 * a;
}
```

Ответ №5

```
#include <iostream>

int doubleNumber(int a)
{
    return 2 * a;
}

int main()
{
    int a;
    std::cout<<"Enter a number: ";
    std::cin>> a;
    std::cout << doubleNumber(a) << std::endl;
    return 0;
}

/*
// Следующее решение является альтернативным:
int main()
{
    int a;
    std::cout<<"Enter a number: ";
    std::cin>> a;
    a = doubleNumber(a);
    std::cout << a << std::endl;
    return 0;
}
*/
```

Примечание: У вас могут быть и другие решения заданий №4 и №5 — это ок. В программировании есть много случаев, когда одну задачу можно решить нескольким способами.